Software Requirements Specification

For

Unbound: Fest Management Platform

Institution: MIT Academy of Engineering (MITAOE)

Project Goal: To architect a centralized, high-concurrency digital ecosystem for college fest management. The system leverages a modern full-stack architecture to ensure data precision, robust security through RBAC, and seamless automated registration workflows.

Contents

1 Introduction **2**
1.1 System Vision 2
1.2 User Hierarchy Access Model 2

2 System Architecture **3**
2.1 The Full-Stack Ecosystem 3

3 Technical Stack Matrix **4**

4 Functional Requirements **5**
4.1 Security Authentication (RBAC) 5
4.2 Registration Logic 5
4.3 Automated Notifications 5

5 Use Case Scenarios: User Interaction **6**
5.1 Use Case 1: Individual Registration 6
5.2 Use Case 2: Team Formation 6

6 Use Case Scenarios: Administration **7**
6.1 Use Case 3: Event Orchestration 7
6.2 Use Case 4: Institutional Oversight 7

7 External Integrations **8**
7.1 Financial Services (Razorpay) 8
7.2 Infrastructure Services (AWS) 8

8 Non-Functional Requirements **9**
8.1 Scalability Performance 9
8.2 Security Reliability 9

9 Conclusion **10**

1 Introduction

Unbound is an enterprise-grade solution designed to address the administrative complexities of organizing large-scale technical fests at MITAOE. The platform replaces manual, error-prone processes with a unified digital backbone.

1.1 System Vision

The engineering vision for Unbound is to provide a "Single Source of Truth" for all fest-related data. By integrating real-time payment verification with automated team orchestration, the system ensures that coordinators can focus on event execution while the platform handles the logistical heavy lifting.

1.2 User Hierarchy Access Model

User Role	Primary Access Right	Core Responsibility
Super Admin	Global System Privileges	Institutional auditing and system-wide configuration.
College Admin	College-Level Privileges	Oversight of departmental fests and budget approvals.
Club Coordinator	Moderator Privileges	Event lifecycle management and participant tracking.
Student	Standard User Privileges	Event discovery and Solo/Team registration flows.

Table 1: User Role Responsibility Matrix

2 System Architecture

Unbound utilizes a **Three-Tier Decoupled Architecture**. This design ensures that the user interface, business logic, and database layers can scale independently, providing maximum uptime during peak registration periods.

2.1 The Full-Stack Ecosystem

The architecture is built on four pillars:

- **Presentation Layer:** A highly responsive interface built with Next.js 16, utilizing Server Components for optimized data delivery.
- **Application Layer:** A robust Spring Boot 3.5.11 backend that manages complex business rules and multi-threaded request processing.
- **Data Persistence:** A PostgreSQL 15 relational database ensuring ACID compliance for all financial and registration records.
- **Cloud Infrastructure:** AWS Fargate providing serverless container orchestration to handle elastic traffic spikes.

3 Technical Stack Matrix

The stack was chosen to balance developer velocity with enterprise-level stability and security.

Component	Technology	Version	Engineering Justification
Language	Java	17	Strict type-safety across all system boundaries.
Backend	Spring Boot	3.5.11	Enterprise-grade transaction and security handling.
Frontend	Next.js	16.x	Hybrid rendering (SSR/SSG) for performance and SEO.
Database	PostgreSQL	15.17	Relational integrity for complex team structures.
Infrastructure	AWS Fargate	Latest	Automated scaling without server management.
Payments	Razorpay SDK	1.4.3	Standardized payment processing and webhooks.

Table 2: Consolidated Technical Stack

4 Functional Requirements

4.1 Security Authentication (RBAC)

The system implements a zero-trust security model. Every request is filtered through a Role-Based Access Control (RBAC) layer that validates signed JWTs (JSON Web Tokens). This ensures that a Club Coordinator cannot access Super Admin audits, and Students are restricted to their own registration data.

4.2 Registration Logic

The core engine supports two primary registration modes:

- **Solo Mode:** Direct student-to-event association finalized upon payment confirmation.
- **Team Mode:** A leader creates a team entity, generating a unique alphanumeric code. Members join via code validation, with the system enforcing strict capacity limits in real-time.

4.3 Automated Notifications

Using SMTP integration, the system triggers automated emails for:

- Successful registration and payment receipts.
- Team Code distribution and member join alerts.

5 Use Case Scenarios: User Interaction

5.1 Use Case 1: Individual Registration

Actor: Student
Flow: Student navigates to the event catalog → Selects a solo event → Redirects to Razorpay gateway → Payment success → System generates unique Registration ID → Confirmation email sent.

5.2 Use Case 2: Team Formation

Actor: Student (Team Leader)
Flow: Leader selects "Create Team" → System generates a unique Team Code → Leader shares code with peers → Peers join through the "Join Team" portal after slot validation.

6 Use Case Scenarios: Administration

6.1 Use Case 3: Event Orchestration

Actor: Club Coordinator

Flow: Coordinator inputs event metadata (fees, date, capacity) → System performs a logic check → Event is published to the live student portal.

6.2 Use Case 4: Institutional Oversight

Actor: Super Admin

Flow: Admin accesses the centralized dashboard → System aggregates real-time data from all fests → Admin exports analytical reports for institutional review.

7 External Integrations

7.1 Financial Services (Razorpay)

The payment flow is secured using HMAC SHA256 signature verification. Webhooks are utilized to ensure that the database is updated only after the payment gateway confirms a successful transaction, preventing "orphan" registrations.

7.2 Infrastructure Services (AWS)

PostgreSQL is hosted on RDS with Multi-AZ deployment for high availability. Static assets and student uploads are managed via S3, ensuring that the application servers remain lightweight and focused on business logic.

8 Non-Functional Requirements

8.1 Scalability Performance

- **Concurrency:** The system must maintain stability under a load of 500-600 concurrent registration attempts.
- **Latency:** Backend API response times must remain below 300ms for 95% of requests.

8.2 Security Reliability

- **Integrity:** All passwords must be hashed using BCrypt with a workload factor of 12.
- **Availability:** The platform target is 99.9% uptime during festival operational hours.
- **Data Safety:** Automated daily backups of the PostgreSQL database are mandatory.

9 Conclusion

Unbound is a comprehensive technical ecosystem tailored for the MITAOE community. By combining the strengths of Spring Boot for secure back-office logic and Next.js 16 for a high-performance frontend, the platform provides a resilient, precise, and scalable solution for fest management. This document serves as the final technical baseline for the production release.